

Module02_Regularization_Nonlinear_Assignment4_abalone

April 5, 2026

- TITLE: Nonlinear Regression Analysis for the Abalone Rings Prediction Problem
- Date: April 05, 2026
- Student Name: Yavo Michel Blaffo
- Course: PREDICTIVE ANALYSIS

1 Imports

```
[1]: import warnings
warnings.filterwarnings("ignore")

from pathlib import Path
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

import statsmodels.api as sm
import statsmodels.formula.api as smf
from statsmodels.stats.anova import anova_lm
from statsmodels.stats.diagnostic import het_breuschpagan
from statsmodels.stats.stattools import jarque_bera, durbin_watson
from statsmodels.stats.outliers_influence import variance_inflation_factor

from sklearn.model_selection import KFold

from IPython.display import display, Image
```

2 File paths and data loading

```
[ ]: from pathlib import Path
import pandas as pd
from IPython.display import display

TRAIN_PATH = Path(r"F:
↳\NU\predictive_analysis\week2\assignment2\playground-series-s4e4\train.csv")
TEST_PATH = Path(r"F:
↳\NU\predictive_analysis\week2\assignment2\playground-series-s4e4\test.csv")
```

```

SAMPLE_SUB_PATH = Path(r"F:
↳\NU\predictive_analysis\week2\assignment2\playground-series-s4e4\sample_submission.
↳csv")

if not TRAIN_PATH.exists():
    raise FileNotFoundError(f"Missing file: {TRAIN_PATH}")
if not TEST_PATH.exists():
    raise FileNotFoundError(f"Missing file: {TEST_PATH}")
if not SAMPLE_SUB_PATH.exists():
    raise FileNotFoundError(f"Missing file: {SAMPLE_SUB_PATH}")

train_raw = pd.read_csv(TRAIN_PATH)
test_raw = pd.read_csv(TEST_PATH)
sample_submission = pd.read_csv(SAMPLE_SUB_PATH)

print("train shape:", train_raw.shape)
print("test shape:", test_raw.shape)
print("sample_submission shape:", sample_submission.shape)

display(train_raw.head())
display(test_raw.head())
display(sample_submission.head())

```

train shape: (90615, 10)

test shape: (60411, 9)

sample_submission shape: (60411, 2)

	id	Sex	Length	Diameter	Height	Whole weight	Whole weight.1	\
0	0	F	0.550	0.430	0.150	0.7715	0.3285	
1	1	F	0.630	0.490	0.145	1.1300	0.4580	
2	2	I	0.160	0.110	0.025	0.0210	0.0055	
3	3	M	0.595	0.475	0.150	0.9145	0.3755	
4	4	I	0.555	0.425	0.130	0.7820	0.3695	

	Whole weight.2	Shell weight	Rings
0	0.1465	0.2400	11
1	0.2765	0.3200	11
2	0.0030	0.0050	6
3	0.2055	0.2500	10
4	0.1600	0.1975	9

	id	Sex	Length	Diameter	Height	Whole weight	Whole weight.1	\
0	90615	M	0.645	0.475	0.155	1.2380	0.6185	
1	90616	M	0.580	0.460	0.160	0.9830	0.4785	
2	90617	M	0.560	0.420	0.140	0.8395	0.3525	
3	90618	M	0.570	0.490	0.145	0.8740	0.3525	
4	90619	I	0.415	0.325	0.110	0.3580	0.1575	

Whole weight.2 Shell weight

0	0.3125	0.3005
1	0.2195	0.2750
2	0.1845	0.2405
3	0.1865	0.2350
4	0.0670	0.1050

	id	Rings
0	90615	10
1	90616	10
2	90617	10
3	90618	10
4	90619	10

3 Cell 3: Clean column names

```
[ ]: def clean_columns(df):
    rename_map = {
        "Sex": "sex",
        "Length": "length",
        "Diameter": "diameter",
        "Height": "height",
        "Whole weight": "whole_weight",
        "Whole weight.1": "whole_weight_1",
        "Whole weight.2": "whole_weight_2",
        "Shell weight": "shell_weight",
        "Rings": "rings",
        "id": "id",
        "Id": "id"
    }
    out = df.rename(columns=rename_map).copy()
    return out

train = clean_columns(train_raw)
test = clean_columns(test_raw)

required_train_cols = [
    "id", "sex", "length", "diameter", "height",
    "whole_weight", "whole_weight_1", "whole_weight_2",
    "shell_weight", "rings"
]

required_test_cols = [
    "id", "sex", "length", "diameter", "height",
    "whole_weight", "whole_weight_1", "whole_weight_2",
    "shell_weight"
]
```

```

missing_train = [c for c in required_train_cols if c not in train.columns]
missing_test = [c for c in required_test_cols if c not in test.columns]

if missing_train:
    raise ValueError(f"Missing columns in train.csv: {missing_train}")
if missing_test:
    raise ValueError(f"Missing columns in test.csv: {missing_test}")

print("Cleaned train columns:", train.columns.tolist())
print("Cleaned test columns:", test.columns.tolist())

```

```

Cleaned train columns: ['id', 'sex', 'length', 'diameter', 'height',
'whole_weight', 'whole_weight_1', 'whole_weight_2', 'shell_weight', 'rings']
Cleaned test columns: ['id', 'sex', 'length', 'diameter', 'height',
'whole_weight', 'whole_weight_1', 'whole_weight_2', 'shell_weight']

```

4 Basic cleaning and validation

```

[ ]: numeric_cols = [
    "length", "diameter", "height",
    "whole_weight", "whole_weight_1",
    "whole_weight_2", "shell_weight"
]

categorical_col = "sex"
target_col = "rings"

# Convert categorical to string just in case
train[categorical_col] = train[categorical_col].astype(str)
test[categorical_col] = test[categorical_col].astype(str)

# Coerce numeric columns
for col in numeric_cols + [target_col]:
    train[col] = pd.to_numeric(train[col], errors="coerce")

for col in numeric_cols:
    test[col] = pd.to_numeric(test[col], errors="coerce")

# Median imputation for any missing numeric values
for col in numeric_cols:
    train[col] = train[col].fillna(train[col].median())
    test[col] = test[col].fillna(train[col].median())

# Mode imputation for sex if needed
train[categorical_col] = train[categorical_col].fillna(train[categorical_col].
    ↪mode().iloc[0])

```

```

test[categorical_col] = test[categorical_col].fillna(train[categorical_col].
↳mode().iloc[0])

# Target must not be missing
train = train.dropna(subset=[target_col]).reset_index(drop=True)

print("Final train shape:", train.shape)
print("Final test shape:", test.shape)

display(train[numeric_cols + [target_col]].describe().T)
display(train[[categorical_col]].describe())

```

Final train shape: (90615, 10)

Final test shape: (60411, 9)

	count	mean	std	min	25%	50%	75%	\
length	90615.0	0.517098	0.118217	0.0750	0.4450	0.5450	0.6000	
diameter	90615.0	0.401679	0.098026	0.0550	0.3450	0.4250	0.4700	
height	90615.0	0.135464	0.038008	0.0000	0.1100	0.1400	0.1600	
whole_weight	90615.0	0.789035	0.457671	0.0020	0.4190	0.7995	1.0675	
whole_weight_1	90615.0	0.340778	0.204428	0.0010	0.1775	0.3300	0.4630	
whole_weight_2	90615.0	0.169422	0.100909	0.0005	0.0865	0.1660	0.2325	
shell_weight	90615.0	0.225898	0.130203	0.0015	0.1200	0.2250	0.3050	
rings	90615.0	9.696794	3.176221	1.0000	8.0000	9.0000	11.0000	

	max
length	0.8150
diameter	0.6500
height	1.1300
whole_weight	2.8255
whole_weight_1	1.4880
whole_weight_2	0.7600
shell_weight	1.0050
rings	29.0000

	sex
count	90615
unique	3
top	I
freq	33093

5 Define formulas

5.1 Baseline linear model is included only for comparison.

5.2 Assignment A.1 focuses on the two nonlinear models.

```
[ ]: linear_formula = (
    "rings ~ C(sex) + length + diameter + height + whole_weight + "
    "whole_weight_1 + whole_weight_2 + shell_weight"
)

# Nonlinear Model 1: Additive polynomial regression with quadratic terms
poly_formula = (
    "rings ~ C(sex) + length + diameter + height + whole_weight + _
    ↪whole_weight_1 + "
    "whole_weight_2 + shell_weight + "
    "I(length**2) + I(diameter**2) + I(height**2) + I(whole_weight**2) + "
    "I(whole_weight_1**2) + I(whole_weight_2**2) + I(shell_weight**2)"
)

# Nonlinear Model 2: Additive natural cubic spline regression
# df values are chosen to give flexibility without being overly unstable
spline_formula = (
    "rings ~ C(sex) + "
    "cr(length, df=5) + "
    "cr(diameter, df=5) + "
    "cr(height, df=4) + "
    "cr(whole_weight, df=5) + "
    "cr(whole_weight_1, df=4) + "
    "cr(whole_weight_2, df=4) + "
    "cr(shell_weight, df=4)"
)

print("Baseline linear formula:")
print(linear_formula)
print("\nPolynomial formula:")
print(poly_formula)
print("\nSpline formula:")
print(spline_formula)
```

Baseline linear formula:

```
rings ~ C(sex) + length + diameter + height + whole_weight + whole_weight_1 +
whole_weight_2 + shell_weight
```

Polynomial formula:

```
rings ~ C(sex) + length + diameter + height + whole_weight + whole_weight_1 +
whole_weight_2 + shell_weight + I(length**2) + I(diameter**2) + I(height**2) +
I(whole_weight**2) + I(whole_weight_1**2) + I(whole_weight_2**2) +
I(shell_weight**2)
```

Spline formula:

```
rings ~ C(sex) + cr(length, df=5) + cr(diameter, df=5) + cr(height, df=4) +  
cr(whole_weight, df=5) + cr(whole_weight_1, df=4) + cr(whole_weight_2, df=4) +  
cr(shell_weight, df=4)
```

6 Helper functions

```
[ ]: def rmsle(y_true, y_pred):  
    y_true = np.asarray(y_true, dtype=float)  
    y_pred = np.asarray(y_pred, dtype=float)  
    y_pred = np.clip(y_pred, 0, None)  
    return float(np.sqrt(np.mean((np.log1p(y_true) - np.log1p(y_pred)) ** 2)))  
  
def regression_metrics(y_true, y_pred):  
    y_true = np.asarray(y_true, dtype=float)  
    y_pred = np.asarray(y_pred, dtype=float)  
  
    rmse = float(np.sqrt(np.mean((y_true - y_pred) ** 2)))  
    mae = float(np.mean(np.abs(y_true - y_pred)))  
  
    ss_res = float(np.sum((y_true - y_pred) ** 2))  
    ss_tot = float(np.sum((y_true - np.mean(y_true)) ** 2))  
    r2 = float(1 - ss_res / ss_tot)  
  
    return {  
        "rmse": rmse,  
        "mae": mae,  
        "r2": r2,  
        "rmsle": rmsle(y_true, y_pred)  
    }  
  
def cross_validate_formula(data, formula, n_splits=5, random_state=42):  
    kf = KFold(n_splits=n_splits, shuffle=True, random_state=random_state)  
  
    fold_rows = []  
  
    for fold, (train_idx, valid_idx) in enumerate(kf.split(data), start=1):  
        fold_train = data.iloc[train_idx].copy()  
        fold_valid = data.iloc[valid_idx].copy()  
  
        model = smf.ols(formula, data=fold_train).fit()  
        preds = model.predict(fold_valid)  
  
        mets = regression_metrics(fold_valid[target_col], preds)  
  
        fold_rows.append({
```

```

        "fold": fold,
        "rmse": mets["rmse"],
        "mae": mets["mae"],
        "r2": mets["r2"],
        "rmsle": mets["rmsle"]
    })

fold_df = pd.DataFrame(fold_rows)

summary_df = pd.DataFrame([
    "cv_rmse_mean": fold_df["rmse"].mean(),
    "cv_rmse_std": fold_df["rmse"].std(ddof=1),
    "cv_mae_mean": fold_df["mae"].mean(),
    "cv_r2_mean": fold_df["r2"].mean(),
    "cv_rmsle_mean": fold_df["rmsle"].mean(),
    "cv_rmsle_std": fold_df["rmsle"].std(ddof=1)
])

return fold_df, summary_df

def fit_model_with_table(model_name, formula, data):
    model = smf.ols(formula, data=data).fit()
    fitted = model.predict(data)
    mets = regression_metrics(data[target_col], fitted)

    row = {
        "model": model_name,
        "n_obs": int(model.nobs),
        "df_model": float(model.df_model),
        "df_resid": float(model.df_resid),
        "r_squared": float(model.rsquared),
        "adj_r_squared": float(model.rsquared_adj),
        "aic": float(model.aic),
        "bic": float(model.bic),
        "f_statistic": float(model.fvalue),
        "f_pvalue": float(model.f_pvalue),
        "train_rmse": mets["rmse"],
        "train_mae": mets["mae"],
        "train_r2": mets["r2"],
        "train_rmsle": mets["rmsle"]
    }

    return model, row

def coefficient_table(model):
    tbl = model.summary2().tables[1].copy()
    tbl = tbl.reset_index().rename(columns={"index": "term"})

```



```

return tbl

def diagnostic_summary_table(model, label):
    resid = model.resid
    exog = model.model.exog

    bp_test = het_breuschpagan(resid, exog)
    jb_stat, jb_pvalue, skewness, kurtosis = jarque_bera(resid)
    dw_stat = durbin_watson(resid)

    out = pd.DataFrame([
        "model": label,
        "breusch_pagan_lm_stat": bp_test[0],
        "breusch_pagan_lm_pvalue": bp_test[1],
        "breusch_pagan_f_stat": bp_test[2],
        "breusch_pagan_f_pvalue": bp_test[3],
        "jarque_bera_stat": jb_stat,
        "jarque_bera_pvalue": jb_pvalue,
        "skewness": skewness,
        "kurtosis": kurtosis,
        "durbin_watson": dw_stat
    ])

    return out

def vif_table(model, top_n=None):
    exog = pd.DataFrame(model.model.exog, columns=model.model.exog_names).copy()

    # Remove intercept
    keep_cols = [c for c in exog.columns if c.lower() not in ("intercept", "const")]
    exog = exog[keep_cols]

    # Remove zero-variance columns if any
    exog = exog.loc[:, exog.nunique() > 1]

    rows = []
    for i, col in enumerate(exog.columns):
        vif_value = variance_inflation_factor(exog.values, i)
        rows.append({"term": col, "VIF": float(vif_value)})

    vif_df = pd.DataFrame(rows).sort_values("VIF", ascending=False).
    reset_index(drop=True)

    if top_n is not None:
        return vif_df.head(top_n)

```

```

    return vif_df

def make_effect_df(train_df, predictor, n_points=200):
    grid = np.linspace(train_df[predictor].min(), train_df[predictor].max(),
        ↪n_points)

    base_row = {
        "sex": train_df["sex"].mode().iloc[0],
        "length": train_df["length"].median(),
        "diameter": train_df["diameter"].median(),
        "height": train_df["height"].median(),
        "whole_weight": train_df["whole_weight"].median(),
        "whole_weight_1": train_df["whole_weight_1"].median(),
        "whole_weight_2": train_df["whole_weight_2"].median(),
        "shell_weight": train_df["shell_weight"].median()
    }

    pred_df = pd.DataFrame([base_row] * n_points)
    pred_df[predictor] = grid
    return pred_df

def interpret_bp(p):
    return "No strong evidence of heteroscedasticity" if p > 0.05 else
        ↪"Evidence of heteroscedasticity"

def interpret_jb(p):
    return "Residuals are closer to normality" if p > 0.05 else "Residuals
        ↪depart from normality"

def interpret_dw(dw):
    if 1.5 <= dw <= 2.5:
        return "No strong evidence of autocorrelation"
    elif dw < 1.5:
        return "Possible positive autocorrelation"
    else:
        return "Possible negative autocorrelation"

```

7 Fit models on full training data

```

[ ]: linear_model, linear_row = fit_model_with_table("Baseline Linear",
        ↪linear_formula, train)
poly_model, poly_row = fit_model_with_table("Nonlinear Model 1: Polynomial",
        ↪poly_formula, train)
spline_model, spline_row = fit_model_with_table("Nonlinear Model 2: Natural
        ↪Cubic Spline", spline_formula, train)

```

```
full_model_table = pd.DataFrame([linear_row, poly_row, spline_row])
display(full_model_table)
```

		model	n_obs	df_model	df_resid	\
0		Baseline Linear	90615	9.0	90605.0	
1		Nonlinear Model 1: Polynomial	90615	16.0	90598.0	
2		Nonlinear Model 2: Natural Cubic Spline	90615	26.0	90588.0	

	r_squared	adj_r_squared	aic	bic	f_statistic	\
0	0.605587	0.605547	382315.117721	382409.261472	15457.325619	
1	0.630001	0.629936	376538.853691	376698.898067	9641.389790	
2	0.632733	0.632628	375887.240607	376141.428733	6002.561014	

	f_pvalue	train_rmse	train_mae	train_r2	train_rmsle
0	0.0	1.994730	1.373854	0.605587	0.164916
1	0.0	1.932006	1.341283	0.630001	0.160628
2	0.0	1.924860	1.332839	0.632733	0.159602

8 Cell 8: 5-fold cross-validation

8.1 This is the main model comparison section for Kaggle alignment.

```
[ ]: linear_cv_folds, linear_cv_summary = cross_validate_formula(train,
    ↪linear_formula, n_splits=5, random_state=42)
poly_cv_folds, poly_cv_summary = cross_validate_formula(train, poly_formula,
    ↪n_splits=5, random_state=42)
spline_cv_folds, spline_cv_summary = cross_validate_formula(train,
    ↪spline_formula, n_splits=5, random_state=42)

linear_cv_summary.insert(0, "model", "Baseline Linear")
poly_cv_summary.insert(0, "model", "Nonlinear Model 1: Polynomial")
spline_cv_summary.insert(0, "model", "Nonlinear Model 2: Natural Cubic Spline")

cv_summary_table = pd.concat([linear_cv_summary, poly_cv_summary,
    ↪spline_cv_summary], ignore_index=True)
cv_summary_table = cv_summary_table.sort_values("cv_rmsle_mean").
    ↪reset_index(drop=True)

print("Cross-validated performance summary:")
display(cv_summary_table)

print("Polynomial fold-by-fold table:")
display(poly_cv_folds)

print("Spline fold-by-fold table:")
display(spline_cv_folds)
```

Cross-validated performance summary:

	model	cv_rmse_mean	cv_rmse_std	\
0	Nonlinear Model 2: Natural Cubic Spline	1.925511	0.021061	
1	Nonlinear Model 1: Polynomial	1.932539	0.020403	
2	Baseline Linear	1.995147	0.022177	

	cv_mae_mean	cv_r2_mean	cv_rmsle_mean	cv_rmsle_std
0	1.332366	0.632472	0.159617	0.001385
1	1.341689	0.629784	0.160674	0.001188
2	1.374050	0.605409	0.164939	0.001460

Polynomial fold-by-fold table:

	fold	rmse	mae	r2	rmsle
0	1	1.962671	1.362476	0.625005	0.162442
1	2	1.923635	1.338778	0.632113	0.161126
2	3	1.938624	1.344999	0.629534	0.160571
3	4	1.930451	1.336295	0.629196	0.159441
4	5	1.907311	1.325896	0.633073	0.159790

Spline fold-by-fold table:

	fold	rmse	mae	r2	rmsle
0	1	1.956419	1.354498	0.627391	0.161583
1	2	1.915745	1.328896	0.635125	0.160162
2	3	1.932816	1.336348	0.631750	0.159666
3	4	1.922814	1.325049	0.632124	0.157999
4	5	1.899763	1.317041	0.635971	0.158674

9 ANOVA comparison for baseline linear vs polynomial

9.1 This helps justify the added nonlinear terms in Model 1.

```
[ ]: anova_linear_vs_poly = anova_lm(linear_model, poly_model)
      display(anova_linear_vs_poly)
```

	df_resid	ssr	df_diff	ss_diff	F	Pr(>F)
0	90605.0	360552.333078	0.0	NaN	NaN	NaN
1	90598.0	338233.841364	7.0	22318.491714	854.020615	0.0

10 Complete statistical table

10.1 Combines full-sample model statistics and CV summary.

```
[ ]: complete_statistical_table = full_model_table.merge(cv_summary_table,
      ↪on="model", how="left")
      complete_statistical_table = complete_statistical_table.
      ↪sort_values("cv_rmsle_mean").reset_index(drop=True)
```

```
print("Complete statistical table:")
display(complete_statistical_table)
```

Complete statistical table:

	model	n_obs	df_model	df_resid	\
0	Nonlinear Model 2: Natural Cubic Spline	90615	26.0	90588.0	
1	Nonlinear Model 1: Polynomial	90615	16.0	90598.0	
2	Baseline Linear	90615	9.0	90605.0	

	r_squared	adj_r_squared	aic	bic	f_statistic	\
0	0.632733	0.632628	375887.240607	376141.428733	6002.561014	
1	0.630001	0.629936	376538.853691	376698.898067	9641.389790	
2	0.605587	0.605547	382315.117721	382409.261472	15457.325619	

	f_pvalue	train_rmse	train_mae	train_r2	train_rmsle	cv_rmse_mean	\
0	0.0	1.924860	1.332839	0.632733	0.159602	1.925511	
1	0.0	1.932006	1.341283	0.630001	0.160628	1.932539	
2	0.0	1.994730	1.373854	0.605587	0.164916	1.995147	

	cv_rmse_std	cv_mae_mean	cv_r2_mean	cv_rmsle_mean	cv_rmsle_std
0	0.021061	1.332366	0.632472	0.159617	0.001385
1	0.020403	1.341689	0.629784	0.160674	0.001188
2	0.022177	1.374050	0.605409	0.164939	0.001460

11 Complete statistical summary table for diagnostics

```
[ ]: linear_diag = diagnostic_summary_table(linear_model, "Baseline Linear")
poly_diag = diagnostic_summary_table(poly_model, "Nonlinear Model 1:
↳Polynomial")
spline_diag = diagnostic_summary_table(spline_model, "Nonlinear Model 2:
↳Natural Cubic Spline")

complete_statistical_summary_table = pd.concat([linear_diag, poly_diag,
↳spline_diag], ignore_index=True)

complete_statistical_summary_table["bp_interpretation"] =
↳complete_statistical_summary_table["breusch_pagan_f_pvalue"].
↳apply(interpret_bp)
complete_statistical_summary_table["jb_interpretation"] =
↳complete_statistical_summary_table["jarque_bera_pvalue"].apply(interpret_jb)
complete_statistical_summary_table["dw_interpretation"] =
↳complete_statistical_summary_table["durbin_watson"].apply(interpret_dw)

print("Complete statistical summary table:")
display(complete_statistical_summary_table)
```

Complete statistical summary table:

	model	breusch_pagan_lm_stat	\		
0	Baseline Linear	8603.949725			
1	Nonlinear Model 1: Polynomial	6633.530221			
2	Nonlinear Model 2: Natural Cubic Spline	6933.382533			

	breusch_pagan_lm_pvalue	breusch_pagan_f_stat	breusch_pagan_f_pvalue	\	
0	0.0	1056.173206	0.0		
1	0.0	447.259804	0.0		
2	0.0	288.677157	0.0		

	jarque_bera_stat	jarque_bera_pvalue	skewness	kurtosis	durbin_watson	\
0	192964.944542	0.0	1.433259	9.549134	1.998794	
1	180219.767528	0.0	1.521883	9.202251	2.002191	
2	178401.372468	0.0	1.512829	9.172213	2.001348	

	bp_interpretation	jb_interpretation	\		
0	Evidence of heteroscedasticity	Residuals depart from normality			
1	Evidence of heteroscedasticity	Residuals depart from normality			
2	Evidence of heteroscedasticity	Residuals depart from normality			

	dw_interpretation
0	No strong evidence of autocorrelation
1	No strong evidence of autocorrelation
2	No strong evidence of autocorrelation

12 Complete coefficient tables

12.1 These are the full regression coefficient tables for deep study.

```
[ ]: poly_coeff_table = coefficient_table(poly_model)
spline_coeff_table = coefficient_table(spline_model)

print("Polynomial model coefficient table:")
display(poly_coeff_table)

print("Spline model coefficient table:")
display(spline_coeff_table)
```

Polynomial model coefficient table:

	term	Coef.	Std.Err.	t	P> t	\
0	Intercept	1.649276	0.110689	14.900106	3.773764e-50	
1	C(sex) [T.I]	-0.471946	0.021550	-21.900247	4.876481e-106	
2	C(sex) [T.M]	-0.017668	0.016248	-1.087408	2.768596e-01	
3	length	16.431468	1.680788	9.776049	1.463880e-22	
4	diameter	-1.551623	2.018950	-0.768529	4.421748e-01	
5	height	19.927382	0.809651	24.612318	2.563750e-133	

6	whole_weight	13.536639	0.324301	41.741011	0.000000e+00
7	whole_weight_1	-30.619501	0.400894	-76.377988	0.000000e+00
8	whole_weight_2	-6.875381	0.836233	-8.221851	2.030024e-16
9	shell_weight	31.006139	0.601818	51.520750	0.000000e+00
10	I(length ** 2)	-24.768897	1.531090	-16.177296	8.819365e-59
11	I(diameter ** 2)	-0.465850	2.322985	-0.200539	8.410593e-01
12	I(height ** 2)	-16.498622	1.355212	-12.174201	4.535434e-34
13	I(whole_weight ** 2)	-3.152756	0.115928	-27.195913	3.271127e-162
14	I(whole_weight_1 ** 2)	14.427365	0.333333	43.282148	0.000000e+00
15	I(whole_weight_2 ** 2)	1.600430	1.487213	1.076127	2.818733e-01
16	I(shell_weight ** 2)	-11.397136	0.721724	-15.791543	4.228167e-56

	[0.025	0.975]
0	1.432327	1.866225
1	-0.514183	-0.429708
2	-0.049513	0.014177
3	13.137139	19.725796
4	-5.508745	2.405500
5	18.340475	21.514290
6	12.901013	14.172265
7	-31.405250	-29.833752
8	-8.514389	-5.236373
9	29.826581	32.185697
10	-27.769818	-21.767975
11	-5.018878	4.087179
12	-19.154824	-13.842420
13	-3.379972	-2.925539
14	13.774036	15.080695
15	-1.314493	4.515354
16	-12.811708	-9.982564

Spline model coefficient table:

	term	Coef.	Std.Err.	t \
0	Intercept	-3.974055e+10	2.942685e+11	-0.135049
1	C(sex) [T.I]	-5.057986e-01	2.519119e-02	-20.078396
2	C(sex) [T.M]	-1.597713e-02	1.622388e-02	-0.984791
3	cr(length, df=5) [0]	1.102405e+11	8.163024e+11	0.135049
4	cr(length, df=5) [1]	1.102405e+11	8.163024e+11	0.135049
5	cr(length, df=5) [2]	1.102405e+11	8.163024e+11	0.135049
6	cr(length, df=5) [3]	1.102405e+11	8.163024e+11	0.135049
7	cr(length, df=5) [4]	1.102405e+11	8.163024e+11	0.135049
8	cr(diameter, df=5) [0]	-1.256805e+10	9.306316e+10	-0.135049
9	cr(diameter, df=5) [1]	-1.256805e+10	9.306316e+10	-0.135049
10	cr(diameter, df=5) [2]	-1.256805e+10	9.306316e+10	-0.135049
11	cr(diameter, df=5) [3]	-1.256805e+10	9.306316e+10	-0.135049
12	cr(diameter, df=5) [4]	-1.256805e+10	9.306316e+10	-0.135049
13	cr(height, df=4) [0]	-1.505031e+10	1.114436e+11	-0.135049
14	cr(height, df=4) [1]	-1.505031e+10	1.114436e+11	-0.135049

15	cr(height, df=4)[2]	-1.505031e+10	1.114436e+11	-0.135049
16	cr(height, df=4)[3]	-1.505031e+10	1.114436e+11	-0.135049
17	cr(whole_weight, df=5)[0]	-8.289373e+09	6.138067e+10	-0.135049
18	cr(whole_weight, df=5)[1]	-8.289373e+09	6.138067e+10	-0.135049
19	cr(whole_weight, df=5)[2]	-8.289373e+09	6.138067e+10	-0.135049
20	cr(whole_weight, df=5)[3]	-8.289373e+09	6.138067e+10	-0.135049
21	cr(whole_weight, df=5)[4]	-8.289373e+09	6.138067e+10	-0.135049
22	cr(whole_weight_1, df=4)[0]	-8.899610e+09	6.589931e+10	-0.135049
23	cr(whole_weight_1, df=4)[1]	-8.899610e+09	6.589931e+10	-0.135049
24	cr(whole_weight_1, df=4)[2]	-8.899610e+09	6.589931e+10	-0.135049
25	cr(whole_weight_1, df=4)[3]	-8.899610e+09	6.589931e+10	-0.135049
26	cr(whole_weight_2, df=4)[0]	-1.318529e+10	9.763366e+10	-0.135049
27	cr(whole_weight_2, df=4)[1]	-1.318529e+10	9.763366e+10	-0.135049
28	cr(whole_weight_2, df=4)[2]	-1.318529e+10	9.763366e+10	-0.135049
29	cr(whole_weight_2, df=4)[3]	-1.318529e+10	9.763366e+10	-0.135049
30	cr(shell_weight, df=4)[0]	-1.250731e+10	9.261343e+10	-0.135049
31	cr(shell_weight, df=4)[1]	-1.250731e+10	9.261343e+10	-0.135049
32	cr(shell_weight, df=4)[2]	-1.250731e+10	9.261343e+10	-0.135049
33	cr(shell_weight, df=4)[3]	-1.250731e+10	9.261343e+10	-0.135049

	P> t	[0.025	0.975]
0	8.925738e-01	-6.165040e+11	5.370229e+11
1	1.787101e-89	-5.551731e-01	-4.564241e-01
2	3.247296e-01	-4.777578e-02	1.582152e-02
3	8.925738e-01	-1.489704e+12	1.710185e+12
4	8.925738e-01	-1.489704e+12	1.710185e+12
5	8.925738e-01	-1.489704e+12	1.710185e+12
6	8.925738e-01	-1.489704e+12	1.710185e+12
7	8.925738e-01	-1.489704e+12	1.710185e+12
8	8.925738e-01	-1.949709e+11	1.698348e+11
9	8.925738e-01	-1.949709e+11	1.698348e+11
10	8.925738e-01	-1.949709e+11	1.698348e+11
11	8.925738e-01	-1.949709e+11	1.698348e+11
12	8.925738e-01	-1.949709e+11	1.698348e+11
13	8.925738e-01	-2.334787e+11	2.033781e+11
14	8.925738e-01	-2.334787e+11	2.033781e+11
15	8.925738e-01	-2.334787e+11	2.033781e+11
16	8.925738e-01	-2.334787e+11	2.033781e+11
17	8.925738e-01	-1.285949e+11	1.120161e+11
18	8.925738e-01	-1.285949e+11	1.120161e+11
19	8.925738e-01	-1.285949e+11	1.120161e+11
20	8.925738e-01	-1.285949e+11	1.120161e+11
21	8.925738e-01	-1.285949e+11	1.120161e+11
22	8.925738e-01	-1.380616e+11	1.202624e+11
23	8.925738e-01	-1.380616e+11	1.202624e+11
24	8.925738e-01	-1.380616e+11	1.202624e+11
25	8.925738e-01	-1.380616e+11	1.202624e+11
26	8.925738e-01	-2.045463e+11	1.781757e+11


```

27 8.925738e-01 -2.045463e+11 1.781757e+11
28 8.925738e-01 -2.045463e+11 1.781757e+11
29 8.925738e-01 -2.045463e+11 1.781757e+11
30 8.925738e-01 -1.940287e+11 1.690141e+11
31 8.925738e-01 -1.940287e+11 1.690141e+11
32 8.925738e-01 -1.940287e+11 1.690141e+11
33 8.925738e-01 -1.940287e+11 1.690141e+11

```

13 VIF tables

14 For the spline model, large VIF values can occur because spline basis functions are related by construction. Interpret with care.

```

[ ]: poly_vif_table = vif_table(poly_model)
    spline_vif_table = vif_table(spline_model)

print("Polynomial model VIF table:")
display(poly_vif_table)

print("Spline model VIF table:")
display(spline_vif_table)

```

Polynomial model VIF table:

	term	VIF
0	diameter	16532.814412
1	length	15575.313662
2	I(diameter ** 2)	4434.087893
3	I(length ** 2)	4175.536126
4	whole_weight	2114.030783
5	whole_weight_2	659.739864
6	whole_weight_1	615.726873
7	shell_weight	594.464957
8	I(whole_weight ** 2)	459.517824
9	height	314.947505
10	I(whole_weight_2 ** 2)	171.205805
11	I(whole_weight_1 ** 2)	148.823947
12	I(shell_weight ** 2)	120.858030
13	I(height ** 2)	24.042083
14	C(sex)[T.I]	4.080087
15	C(sex)[T.M]	2.178851

Spline model VIF table:

	term	VIF
0	cr(length, df=5)[0]	inf
1	cr(length, df=5)[4]	inf

```

2         cr(length, df=5)[1]          inf
3         cr(length, df=5)[2]          inf
4         cr(length, df=5)[3]          inf
5     cr(whole_weight, df=5)[4]  3.283054e+09
6     cr(shell_weight, df=4)[3]  2.494198e+09
7         cr(diameter, df=5)[1]  9.284804e+08
8         cr(height, df=4)[3]  8.076181e+08
9         cr(diameter, df=5)[0]  6.898638e+08
10    cr(whole_weight, df=5)[3]  3.958429e+08
11 cr(whole_weight_2, df=4)[3]  2.448414e+08
12 cr(whole_weight_1, df=4)[3]  1.649204e+08
13 cr(whole_weight_2, df=4)[0]  1.292563e+08
14         cr(diameter, df=5)[4]  1.173365e+08
15 cr(whole_weight_1, df=4)[0]  1.065917e+08
16    cr(whole_weight, df=5)[1]  1.003533e+08
17    cr(shell_weight, df=4)[2]  7.336806e+07
18    cr(whole_weight, df=5)[0]  5.132931e+07
19         cr(height, df=4)[0]  3.134750e+07
20         cr(height, df=4)[1]  2.757092e+07
21    cr(shell_weight, df=4)[0]  2.686157e+07
22         cr(diameter, df=5)[3]  2.332472e+07
23 cr(whole_weight_1, df=4)[2]  1.599109e+07
24 cr(whole_weight_2, df=4)[2]  1.181608e+07
25         cr(diameter, df=5)[2]  9.429161e+06
26    cr(shell_weight, df=4)[1]  6.174544e+06
27 cr(whole_weight_2, df=4)[1]  2.877571e+06
28    cr(whole_weight, df=5)[2]  2.564108e+06
29         cr(height, df=4)[2]  2.215053e+06
30 cr(whole_weight_1, df=4)[1]  1.539212e+06
31         C(sex)[T.I]  2.680506e+00
32         C(sex)[T.M]  1.444498e+00

```

15 Residual and diagnostic plots

```

[ ]: def diagnostic_plots(model, model_name, qq_sample_size=5000):
    fitted = model.fittedvalues
    resid = model.resid

    fig, axes = plt.subplots(2, 2, figsize=(14, 10))

    # Residuals vs fitted
    axes[0, 0].scatter(fitted, resid, alpha=0.25, s=12)
    axes[0, 0].axhline(0, color="red", linestyle="--")
    axes[0, 0].set_title(f"{model_name}: Residuals vs Fitted")
    axes[0, 0].set_xlabel("Fitted values")
    axes[0, 0].set_ylabel("Residuals")

```

```

# Actual vs fitted
axes[0, 1].scatter(model.model.endog, fitted, alpha=0.25, s=12)
min_val = min(model.model.endog.min(), fitted.min())
max_val = max(model.model.endog.max(), fitted.max())
axes[0, 1].plot([min_val, max_val], [min_val, max_val], color="red",
↪linestyle="--")
axes[0, 1].set_title(f"{model_name}: Actual vs Fitted")
axes[0, 1].set_xlabel("Actual Rings")
axes[0, 1].set_ylabel("Fitted Rings")

# Histogram of residuals
axes[1, 0].hist(resid, bins=40, alpha=0.8)
axes[1, 0].set_title(f"{model_name}: Residual Histogram")
axes[1, 0].set_xlabel("Residual")
axes[1, 0].set_ylabel("Frequency")

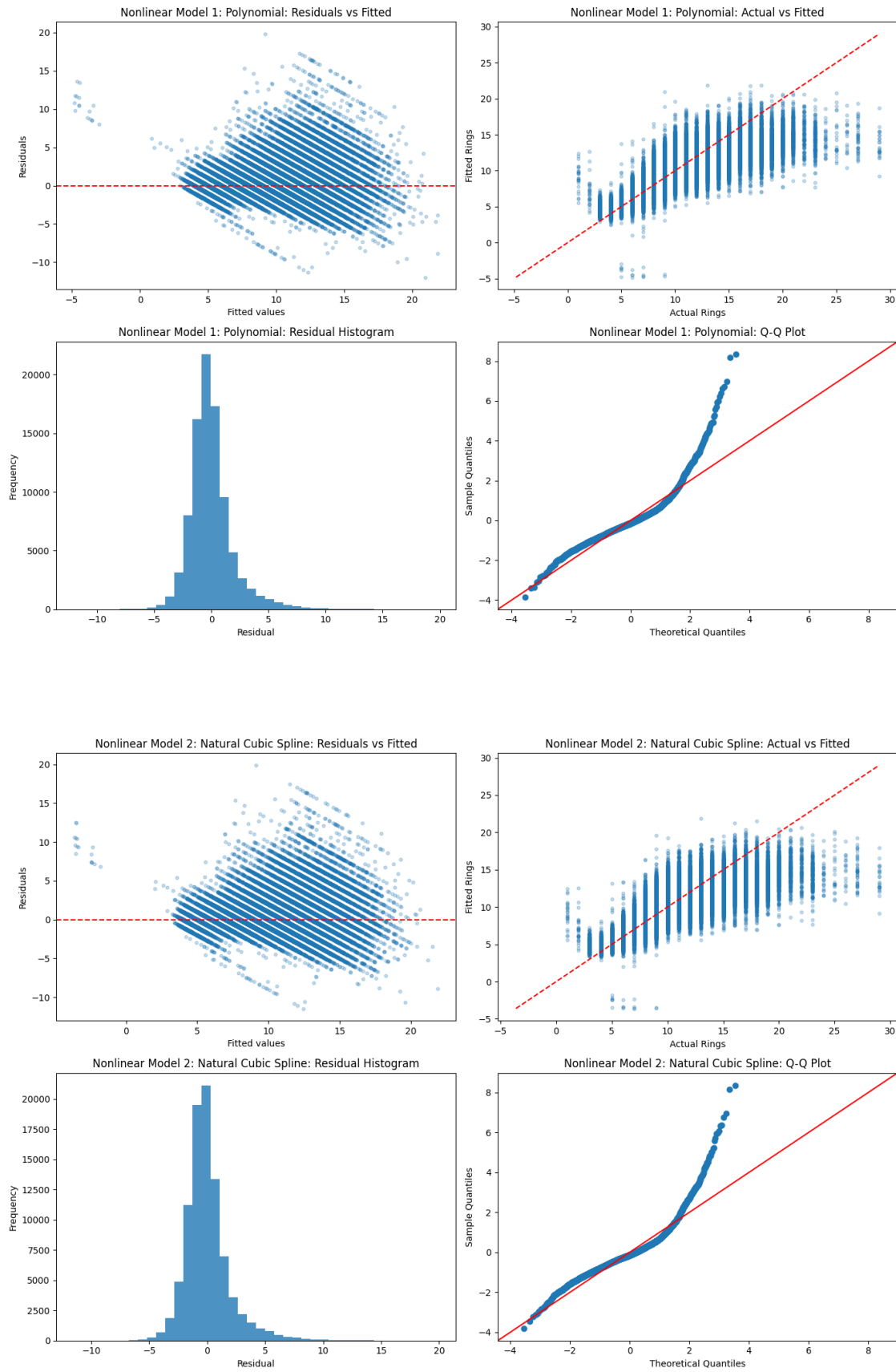
# QQ plot using a sample for readability on large data
qq_resid = resid
if len(qq_resid) > qq_sample_size:
    qq_resid = qq_resid.sample(qq_sample_size, random_state=42)

sm.qqplot(qq_resid, line="45", ax=axes[1, 1], fit=True)
axes[1, 1].set_title(f"{model_name}: Q-Q Plot")

plt.tight_layout()
plt.show()

diagnostic_plots(poly_model, "Nonlinear Model 1: Polynomial")
diagnostic_plots(spline_model, "Nonlinear Model 2: Natural Cubic Spline")

```



16 Informative nonlinear effect plots

16.1 These help justify the presence of nonlinearity in the data.

```
[ ]: selected_predictors = ["length", "diameter", "whole_weight"]

fig, axes = plt.subplots(1, 3, figsize=(18, 5))

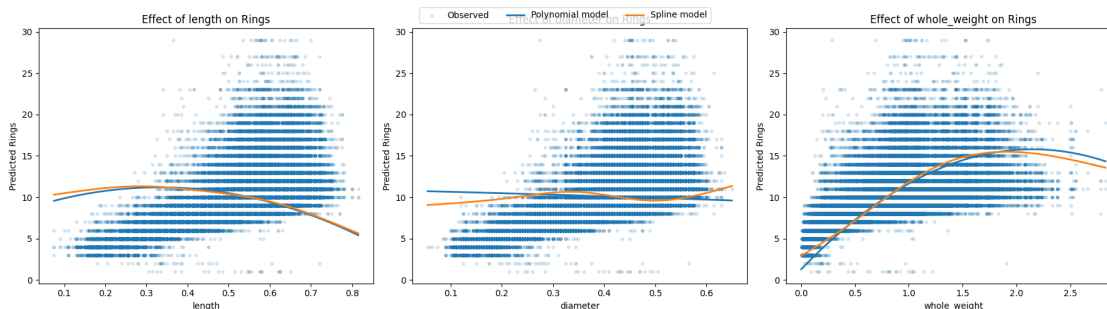
for ax, predictor in zip(axes, selected_predictors):
    pred_df = make_effect_df(train, predictor, n_points=250)

    poly_pred = poly_model.predict(pred_df)
    spline_pred = spline_model.predict(pred_df)

    ax.scatter(train[predictor], train[target_col], alpha=0.15, s=10,
    ↪label="Observed")
    ax.plot(pred_df[predictor], poly_pred, linewidth=2, label="Polynomial
    ↪model")
    ax.plot(pred_df[predictor], spline_pred, linewidth=2, label="Spline model")

    ax.set_title(f"Effect of {predictor} on Rings")
    ax.set_xlabel(predictor)
    ax.set_ylabel("Predicted Rings")

handles, labels = axes[0].get_legend_handles_labels()
fig.legend(handles, labels, loc="upper center", ncol=3)
plt.tight_layout()
plt.show()
```



17 Choose the best model by CV RMSLE

```
[ ]: best_model_name = cv_summary_table.iloc[0]["model"]

if best_model_name == "Baseline Linear":
    best_model = linear_model
    best_formula = linear_formula
elif best_model_name == "Nonlinear Model 1: Polynomial":
    best_model = poly_model
    best_formula = poly_formula
else:
    best_model = spline_model
    best_formula = spline_formula

print("Best model selected by lowest mean CV RMSLE:")
print(best_model_name)
print("\nFormula used:")
print(best_formula)
```

Best model selected by lowest mean CV RMSLE:
Nonlinear Model 2: Natural Cubic Spline

Formula used:

```
rings ~ C(sex) + cr(length, df=5) + cr(diameter, df=5) + cr(height, df=4) +  
cr(whole_weight, df=5) + cr(whole_weight_1, df=4) + cr(whole_weight_2, df=4) +  
cr(shell_weight, df=4)
```

18 Assignment-ready interpretation text

18.1 This prints direct interpretation sentences for A.2 and A.3.

```
[ ]: poly_bp_p = float(poly_diag["breusch_pagan_f_pvalue"].iloc[0])
poly_jb_p = float(poly_diag["jarque_bera_pvalue"].iloc[0])
poly_dw = float(poly_diag["durbin_watson"].iloc[0])

spline_bp_p = float(spline_diag["breusch_pagan_f_pvalue"].iloc[0])
spline_jb_p = float(spline_diag["jarque_bera_pvalue"].iloc[0])
spline_dw = float(spline_diag["durbin_watson"].iloc[0])

print("INTERPRETATION FOR A.1")
print("-" * 60)
print("Two nonlinear models were fitted:")
print("1. An additive polynomial regression model with quadratic terms.")
print("2. An additive natural cubic spline regression model.")
print()
print("The main comparison metric was cross-validated RMSLE because that is  
↪aligned with the Kaggle competition metric.")
```

```

print(f"The best model by mean CV RMSLE is: {best_model_name}")
print()

print("INTERPRETATION FOR A.2")
print("-" * 60)
print("The coefficient tables above provide the full statistical output for_
↳both nonlinear models.")
print("The polynomial model is easier to interpret because each squared term_
↳shows whether curvature is present.")
print("The spline model is more flexible, so it is usually interpreted through_
↳fitted curves and effect plots instead of single coefficients.")
print("The effect plots above show whether the relationship between Rings and_
↳each predictor bends or changes slope.")
print()

print("INTERPRETATION FOR A.3")
print("-" * 60)
print("Polynomial model assumptions:")
print(f"- Homoscedasticity: {interpret_bp(poly_bp_p)} (Breusch-Pagan p-value =_
↳{poly_bp_p:.6f})")
print(f"- Normality: {interpret_jb(poly_jb_p)} (Jarque-Bera p-value =_
↳{poly_jb_p:.6f})")
print(f"- Independence: {interpret_dw(poly_dw)} (Durbin-Watson = {poly_dw:.
↳4f})")
print()
print("Spline model assumptions:")
print(f"- Homoscedasticity: {interpret_bp(spline_bp_p)} (Breusch-Pagan p-value_
↳= {spline_bp_p:.6f})")
print(f"- Normality: {interpret_jb(spline_jb_p)} (Jarque-Bera p-value =_
↳{spline_jb_p:.6f})")
print(f"- Independence: {interpret_dw(spline_dw)} (Durbin-Watson = {spline_dw:.
↳4f})")
print()
print("The VIF tables should be used to check multicollinearity.")
print("High VIF values in the spline model are common because spline basis_
↳functions are mathematically related.")
print("For the polynomial model, very high VIF values indicate stronger_
↳collinearity among the raw and squared terms.")
print()
print("Final modeling conclusion:")
print(f"- Based on the lowest cross-validated RMSLE, the preferred model for_
↳Kaggle submission is: {best_model_name}")

```

INTERPRETATION FOR A.1

Two nonlinear models were fitted:

1. An additive polynomial regression model with quadratic terms.

2. An additive natural cubic spline regression model.

The main comparison metric was cross-validated RMSLE because that is aligned with the Kaggle competition metric.

The best model by mean CV RMSLE is: Nonlinear Model 2: Natural Cubic Spline

INTERPRETATION FOR A.2

The coefficient tables above provide the full statistical output for both nonlinear models.

The polynomial model is easier to interpret because each squared term shows whether curvature is present.

The spline model is more flexible, so it is usually interpreted through fitted curves and effect plots instead of single coefficients.

The effect plots above show whether the relationship between Rings and each predictor bends or changes slope.

INTERPRETATION FOR A.3

Polynomial model assumptions:

- Homoscedasticity: Evidence of heteroscedasticity (Breusch-Pagan p-value = 0.000000)
- Normality: Residuals depart from normality (Jarque-Bera p-value = 0.000000)
- Independence: No strong evidence of autocorrelation (Durbin-Watson = 2.0022)

Spline model assumptions:

- Homoscedasticity: Evidence of heteroscedasticity (Breusch-Pagan p-value = 0.000000)
- Normality: Residuals depart from normality (Jarque-Bera p-value = 0.000000)
- Independence: No strong evidence of autocorrelation (Durbin-Watson = 2.0013)

The VIF tables should be used to check multicollinearity.

High VIF values in the spline model are common because spline basis functions are mathematically related.

For the polynomial model, very high VIF values indicate stronger collinearity among the raw and squared terms.

Final modeling conclusion:

- Based on the lowest cross-validated RMSLE, the preferred model for Kaggle submission is: Nonlinear Model 2: Natural Cubic Spline

19 Predict on test data

19.1 Predictions are clipped to the training target range to avoid unreasonable values.

```
[ ]: test_predictions = best_model.predict(test)

# Clip predictions to observed training target range
train_min = train[target_col].min()
train_max = train[target_col].max()
test_predictions = np.clip(test_predictions, train_min, train_max)

print("Prediction summary:")
print(pd.Series(test_predictions).describe())
```

```
Prediction summary:
count      60411.000000
mean         9.707532
std          2.524808
min          1.000000
25%          7.913182
50%          9.633793
75%         11.300283
max         21.390293
dtype: float64
```

20 Part C: Nonlinear Regression Analysis for the Abalone Rings Prediction Problem

20.1 (a) Building Two Nonlinear Models

In our Abalone dataset analysis collected from Kaggle, we were asked to construct and evaluate two nonlinear regression models. Our target variable was Rings, which was used to predict abalone age. The training dataset contained 90.615 observations and 10 variables, while the test dataset contained 60.411 observations and 9 variables. The first nonlinear model was an additive polynomial regression model. The response variable Rings was modeled as a function of Sex, all original quantitative predictors, and a quadratic term for each quantitative predictor. Polynomial regression extends linear regression by adding transformed versions of the original values, such as X^2 , while the model remains linear in the unknown coefficients and can still be estimated with OLS. Here, the goal was not to force every predictor into a straight line relationship with Rings, but instead to follow the data to show whether curvature was present (Chesnaye et al., 2024). Our results showed that the polynomial model improved fit relative to the baseline linear model. The baseline linear model produced an R^2 of 0.605587, a training RMSE of 1.994730, and a training RMSLE of 0.164916. By contrast, the polynomial model increased R^2 to 0.630001, reduced training RMSE to 1.932006, and lowered training RMSLE to 0.160628. This means the nonlinear terms captured patterns that the baseline linear model could not represent. The ANOVA comparison between the linear and polynomial models further confirmed this improvement. When it comes to the test dataset, we have the metrics like F-statistics of 854.020615 with a p-value of 0.0, and this shows that the added

quadratic terms significantly improved the model. The second nonlinear model was an additive natural cubic spline regression, which replaced straight line effects with smooth spline functions for each quantitative predictor while keeping the additive structure of the regression (James et al., 2023). The spline model produced the best overall performance, full training data has and R2 of 0.632733, a training RMSE of 1.924860, and a training RMSLE of 0.159602, which are better than both polynomial and the baseline linear models, and the five cross validation confirms this assertion with a mean cross validated RMSE equal 1.925511, cross validated MAE equal 1.332366, mean cross validated R2 equal 0.632472, mean cross validated RMSLE was 0.159617. The polynomial model was close, kind of slightly worse, with a mean cross-validated RMSLE of 0.160674. The baseline linear model was definitely weaker, with a mean cross-validated RMSLE of 0.164939. Based on this body of evidence, we can say that the preferred non-linear model for this analysis was the natural cubic spline model. The plots also showed that `whole_weight` has a strong curved relationship with Rings. Length also showed a rise followed by a decline. Diameter curvature was more modest, but the relationship still did not appear perfectly linear (De Giosa et al., 2025).

20.2 (b) Interpretation of the Regression Models and Kaggle Submission Evidence

The interpretation of this analysis is pretty straightforward. The polynomial regression model is the easier of the two nonlinear models to interpret. In that model, $C(\text{sex})[T.I]$ had a negative and statistically significant coefficient, which indicates that, holding the other predictors constant, observations in the I category tended to have lower predicted Rings than the reference group. The coefficient for $C(\text{sex})[T.M]$ was not statistically significant, which suggests that after controlling for the other predictors, the distinction between that group and the reference group was weak in this model. The coefficient for $I(\text{length}^2)$ was negative and significant, which indicates that the effect of length was not simply increasing across the entire range. Instead, the relationship rose and then turned downward, which is consistent with the effect plot (James et al., 2023). The same pattern appeared for height, `whole_weight`, and `shell_weight`, where the squared terms were negative and statistically significant. That pattern suggests diminishing returns and curvature rather than a constant linear increase. The model also showed a strong negative linear coefficient for `whole_weight_1` and a strong positive squared coefficient for $I(\text{whole_weight_12})$, which indicates a more complex curved response. Some terms, such as diameter, $I(\text{diameter}^2)$, and $I(\text{whole_weight_22})$, were not statistically significant, meaning their independent contribution was weaker once the other measurements were included. The VIF table showed extremely high multicollinearity for several predictors, especially diameter, length, and their squared terms. This is expected because raw size variables are highly related to one another, and squaring them creates even stronger dependence. As a result, the polynomial model is best interpreted through its overall fitted pattern rather than by treating each coefficient as a clean and isolated effect (Chesnaye et al., 2024). The spline model must be interpreted differently. Its basis coefficients were extremely large and not practically meaningful on their own. That is not unusual for spline basis representations. The spline effect plots showed that `whole_weight` had the clearest nonlinear association with Rings, with steep gains at lower and middle levels and flattening at the upper end. Length also showed a pronounced bend, while diameter displayed milder curvature. Because the model is additive, each smooth curve can be interpreted as the isolated contribution of one predictor while the others are held fixed. The supplied outputs show that the selected best model was the natural cubic spline model (De Giosa et al., 2025).

20.3 (c) Investigation of Assumptions and Interpretation of Findings

The first assumption issue examined was whether a simple linear structure was adequate. The comparison results strongly suggest that it was not. Both nonlinear models outperformed the baseline linear model on training fit and on cross-validated error. The ANOVA comparison between the linear and polynomial models was highly significant, which confirmed that adding nonlinear terms materially improved the model. The effect plots also showed clear bends in the relationships for length and whole weight. These findings support the conclusion that the linearity assumption was too restrictive for this problem and that nonlinear modeling was justified (Chesnaye et al., 2024). The second assumption issue was homoscedasticity, or constant error variance. Both nonlinear models showed evidence against this assumption. For the polynomial model, the Breusch Pagan F test p-value was 0.0. For the spline model, the Breusch Pagan F test p-value was also 0.0. These results indicate heteroscedasticity in both models. The residual versus fitted plots support the same conclusion. The spread of residuals was not constant across fitted values. The variability widened in some parts of the fitted range and narrowed in others. This means that prediction error changes across the response scale rather than remaining uniform. As a result, the constant variance assumption was violated (Chesnaye et al., 2024). The third assumption issue was residual normality. Once again, both nonlinear models failed this assumption. The Jarque-Bera p-values were 0.0 for both models, which means the residuals departed from normality. The residual histograms were centered near zero, but they were visibly right-skewed and had heavy tails. The QQ plots reinforced this conclusion. In both models, the residuals followed the reference line in the middle only loosely, but they bent away from the line in the tails, especially in the upper tail. That pattern indicates skewness and excess kurtosis. The reported skewness values of 1.521883 for the polynomial model and 1.512829 for the spline model, along with kurtosis values above 9, confirm that the residual distributions were not close to normal (Chesnaye et al., 2024).

The fourth assumption issue was the independence of errors. On this point, the results were favorable. The Durbin Watson statistic was 2.002191 for the polynomial model and 2.001348 for the spline model. Values near 2 suggest no strong evidence of autocorrelation. Therefore, based on the standard interpretation of Durbin-Watson, the independence assumption did not appear to be seriously violated in either nonlinear model (James et al., 2023). Another important issue was multicollinearity. The polynomial model showed extremely large VIF values. Diameter and length had VIF values above 15,000, while several other predictors and squared terms were also very large. This means that the predictors were strongly correlated with one another. In a body measurement dataset, this is not surprising because many size variables reflect overlapping biological dimensions. The spline model also showed huge and even infinite VIF values, but those should be interpreted differently. In spline regression, the basis functions are mathematically related by construction. Therefore, very large VIF values are expected and do not automatically imply that the model is unusable (De Giosa et al., 2025). The nonlinear models successfully addressed the problem of inadequate linearity and improved prediction accuracy. They did not fully satisfy the constant variance and normality assumptions, but they did show acceptable independence of errors. The polynomial model was easier to explain in direct algebraic terms, but it suffered from substantial multicollinearity. The natural cubic spline model was harder to interpret coefficient by coefficient, yet it captured the predictor response shapes more flexibly and produced the best predictive results. For that reason, the spline model is the strongest final choice for the Kaggle competition portion of the assignment (James et al., 2023).

21 References

- Chesnaye, N. C., van Diepen, M., Dekker, F., Zoccali, C., Jager, K. J., & Stel, V. S. (2024). Non-linear relationships in Clinical Research. *Nephrology Dialysis Transplantation*, 40(2), 244–254. <https://doi.org/10.1093/ndt/gfae187>
- De Giosa, M., Falini, A., Iavernaro, F., Losito, S., Mazzia, F., & Rubino, G. (2025). A robust variant of cubic smoothing spline approximation. *Numerical Algorithms*, 101(1), 221–241. <https://doi.org/10.1007/s11075-024-02003-7>
- James, G., Witten, D., Hastie, T., Tibshirani, R., & Taylor, J. (2023). An introduction to statistical learning with applications in Python. Springer Nature.
- Lin, Y.-S., Fan, C.-P., Lee, M.-Y., & Lee, Y.-H. (2026). Mathematical computation of piecewise linear regression with endogenous segmentation for accurate data-based Model Building: An example of the Phillips curve. *Mathematics*, 14(6), 1041. <https://doi.org/10.3390/math14061041>